

Statistics 367-1-4361

Lecture 11: Correlation and Repeated Measures

Opher Donchin

< **Select another question**



Go to wooclap.com

Enter the event code in the top banner

Event code

NEMGOJY



Click on the projected screen to start the question

 **Copy participation link**

wooclap



100%



11.1 The Correlation Coefficient

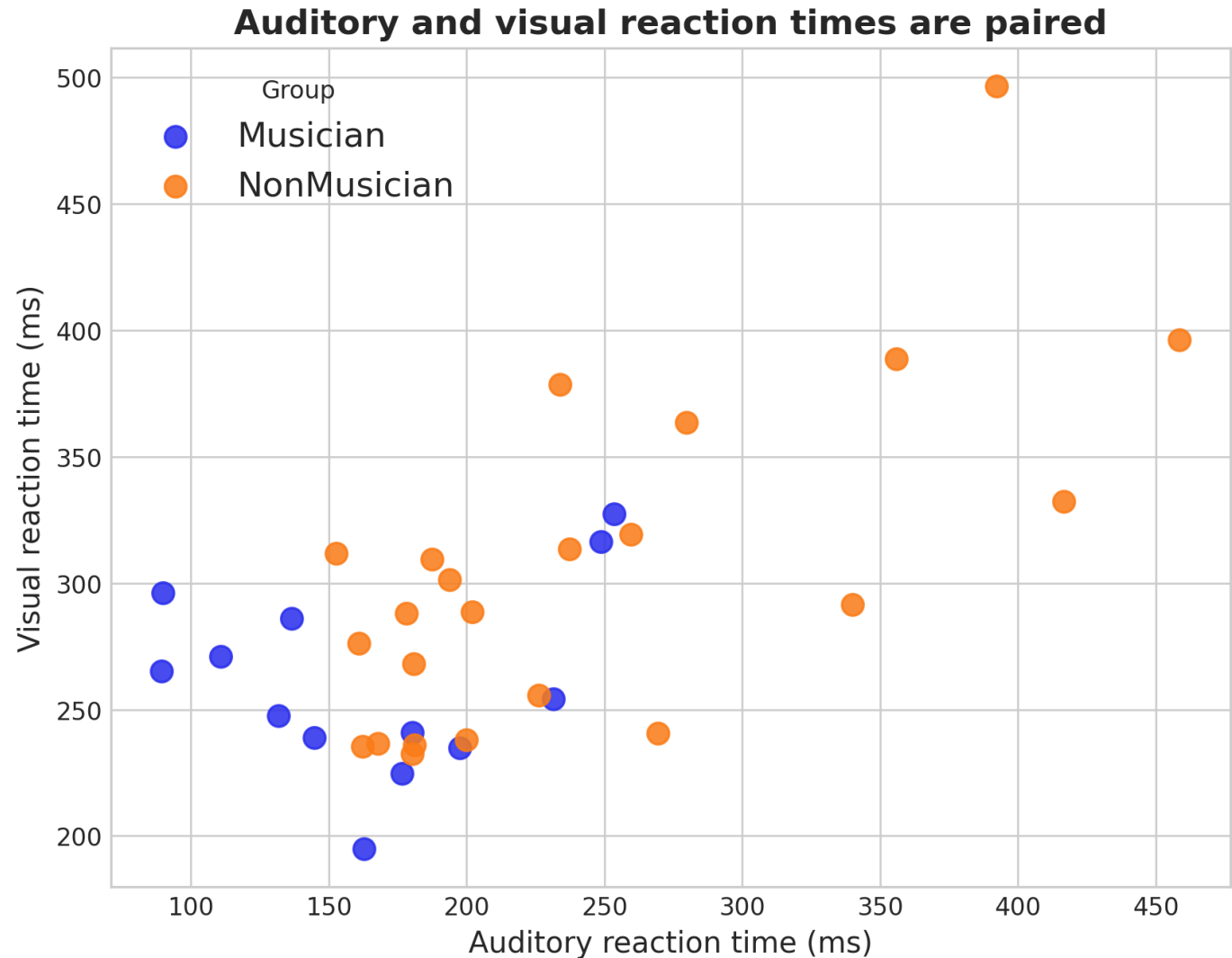
From sample association to population correlation in a likelihood

Correlation and causation

- Regression is a causal model
 - One variable explains another
 - Independent
 - Under our control
- Many relationships aren't causal
 - Reaction times on different task types
 - Volumes of different brain areas
 - Systolic and diastolic blood pressure
 - Resting heart rate and heart-rate variability
- A non-causal relationship is a correlation

Correlation is symmetric

- Auditory and visual reaction times
 - Axes can be swapped
 - Both variables are measures with noise



Population correlation coefficient ρ

$$\rho = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$

$$\sigma_{xy} = E\left((x - \mu_x)(y - \mu_y)\right)$$

$$\sigma_x^2 = \sqrt{E\left((x - \mu_x)^2\right)}$$

- $\rho \in (-1, 1)$
 - Sign is direction
 - Magnitude is strength
- $\rho = 0$
 - No linear association
- $|\rho| = 1$
 - Perfect linear relationship
- Like all parameters, it can have a prior and posterior distribution

Population correlation ρ

$$r = \frac{s_{xy}}{s_x s_y}$$

$$s_{xy} = \sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})$$
$$s_x^2 = \sum_{i=1}^N (x_i - \bar{x})^2$$

- r is an estimate of ρ
 - Called the correlation coefficient
 - Also Pearson's r
 - Also product moment correlation
 - Maximum likelihood estimator
 - Like all estimates, it has a sampling distribution

ρ^2 as a measure of effect size

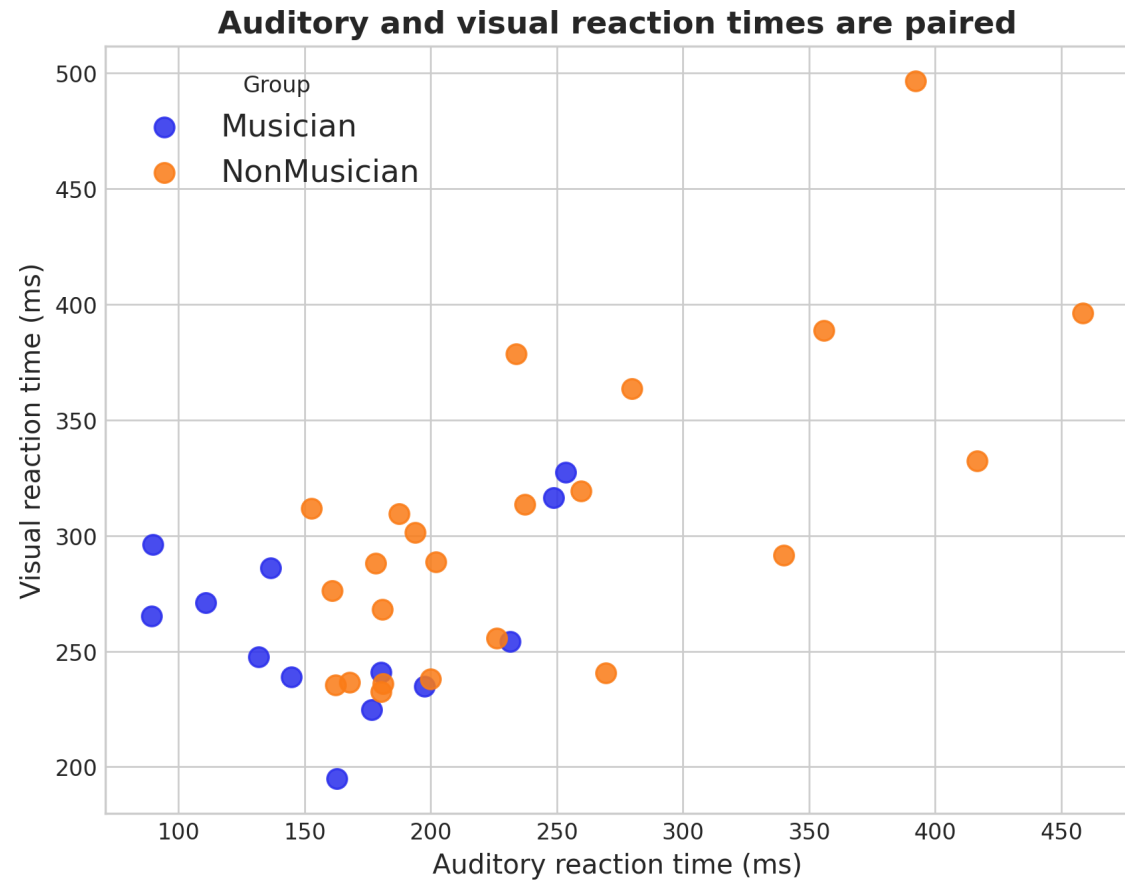
ρ^2 = the fraction of variance in Y shared with X (and vice versa)

- Correlation sizes vary substantially between fields
 - Different measures of lung volume: $\rho = 0.96$, $\rho^2 = 0.92$
 - Strength measures in older adults: $\rho = 0.5$, $\rho^2 = 0.25$
 - Anatomical to functional measures: $\rho = 0.23$, $\rho^2 = 0.05$
- Because of these differences
 - Hard to set a general ROPE for correlations
 - Despite it being a normalized measure

ρ^2	Effect size
0.01	Small
0.09	Medium
0.25	Large

r and r^2 for the correlation of reaction times

- $r = 0.67$
- $r^2 = 0.45$



We model correlation using the multivariate normal

$$\begin{pmatrix} x \\ y \end{pmatrix} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad \boldsymbol{\mu} = \begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix} \quad \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{pmatrix}$$

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{k/2} |\boldsymbol{\Sigma}|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$

- The multivariate normal likelihood
 - Mean and variance for each variable
 - Covariance for each pair of variables
 - Simplest, most convenient likelihood with these parameters

Correlation is normalized covariance

$$\begin{pmatrix} x \\ y \end{pmatrix} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{pmatrix}$$

$$\rho = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$

$$\sigma_{xy} = \rho \sigma_x \sigma_y$$

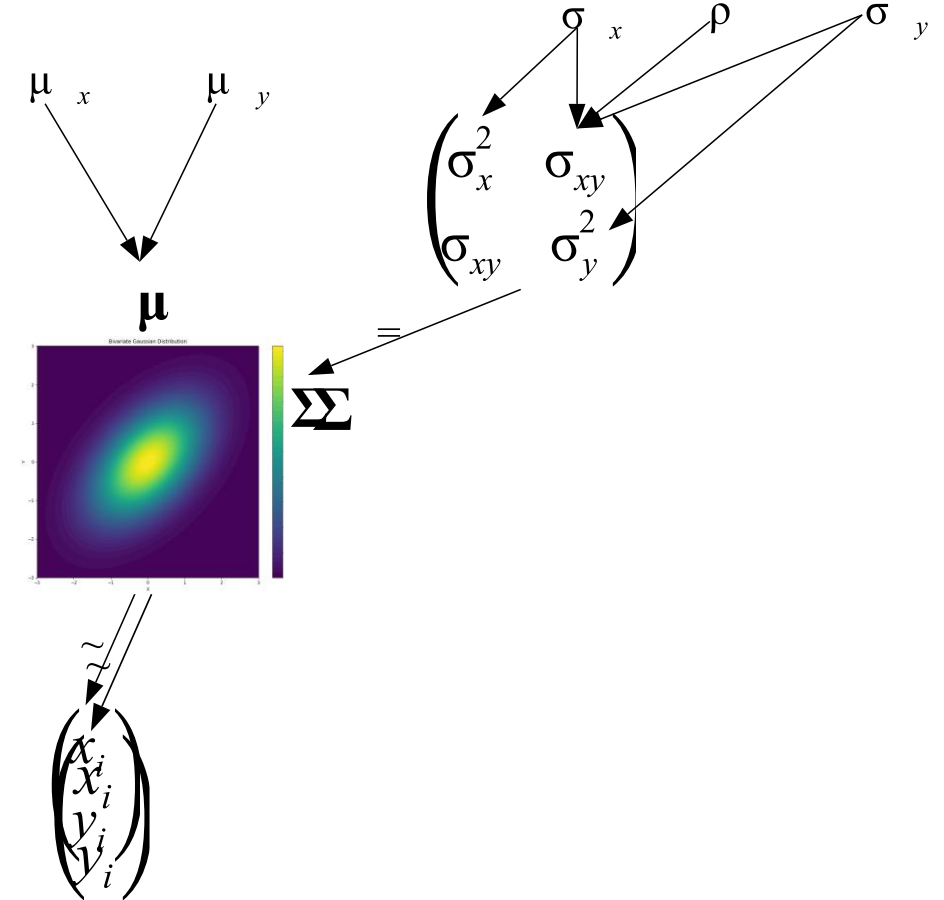
- The covariance matrix
 - Variance on the diagonal
 - Covariance on the off diagonals
 - Symmetric
- Can build the covariance out of correlation and variances

How do we model correlations?

$$\begin{pmatrix} x \\ y \end{pmatrix} \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

$$\boldsymbol{\mu} = \begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix} \quad \boldsymbol{\Sigma} = \begin{pmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{pmatrix} \quad \sigma_{xy} = \sigma_x \sigma_y \rho$$

- We want to estimate ρ .
 - For Bayesian inference, ρ must appear inside a likelihood.
 - We need a likelihood that describes a pair of outcomes jointly
- Each participant contributes a vector observation
 - (auditory RT, visual RT)
- The bivariate normal distribution gives us this
 - Our first multivariate likelihood



< Select another question

Edit question

Go to wooclap.com and use the code **NEMGOJY**



Which of the following would be a *poor* fit to a bivariate normal likelihood to reveal



- ① Asking whether the first variable has a larger variance than the second
- ② Asking whether the first variable has a larger covariance than the second
- ③ Asking whether the first variable has a larger covariance than the second
- ④ Asking whether the first variable has a smaller variance than the second

Click on the projected screen to start the question

wooclap



100%



11.2 Two-group bivariate normal model

Estimating and comparing correlations separately by group

The shape of the data is multivariate

```
audiovisual = audiovisual_wide[["auditory_z", "visual_z"]].to_numpy()  
group_index = audiovisual_wide["group_idx"].to_numpy()
```

```
nonmusician_index = group_order.index("NonMusician")  
musician_index = group_order.index("Musician")
```

```
audiovisual_coords = {  
    "subject": np.arange(len(audiovisual)),  
    "group": group_order,  
    "rt_var": ["auditory_z", "visual_z"],  
    "rt_var_cov": ["auditory_z", "visual_z"],  
}
```

- Data

- `audiovisual` : reaction time data ($N \times 2$)
- `group_index` : group (0,1) for each participant ($N \times 1$)

- Coordinates

- `subject` : $[1:N]$
- `group` : ["NonMusician", "Musician"]
- `rt_var` and `rt_var_cov` : ["auditory", "visual"]

μ , σ are indexed by group and variables

```
audiovisual_coords = {  
    "subject": np.arange(len(audiovisual)),  
    "group": group_order,  
    "rt_var": ["auditory_z", "visual_z"],  
    "rt_var_cov": ["auditory_z", "visual_z"],  
}
```

```
mu = pm.Normal(  
    "mu",  
    mu=mu_mu_correlation,  
    sigma=sigma_mu_correlation,  
    dims=("group", "rt_var"),  
)  
sigma = pm.HalfNormal(  
    "sigma",  
    sigma=sigma_sigma_correlation,  
    dims=("group", "rt_var"),  
)
```

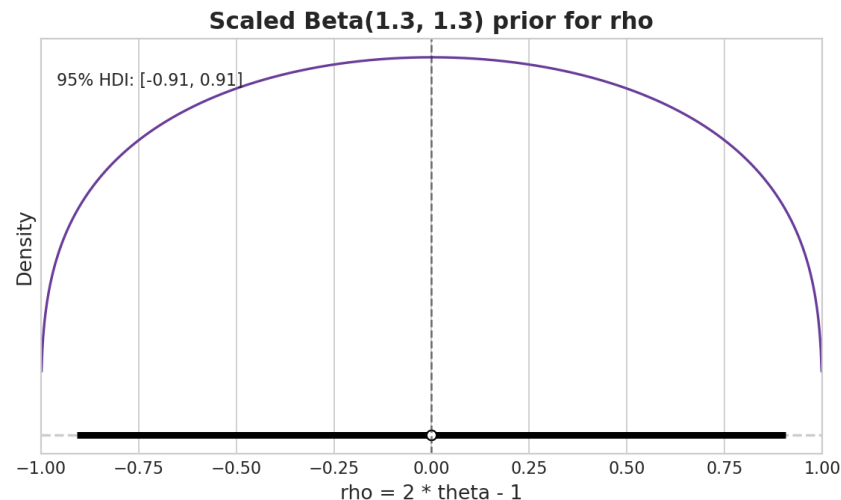
- Both μ and σ are 2×2
 - Two groups
 - Musician and non-musician
 - Two variables
 - Auditory and visual

ρ is only indexed by group

```
audiovisual_coords = {  
    "subject": np.arange(len(audiovisual)),  
    "group": group_order,  
    "rt_var": ["auditory_z", "visual_z"],  
    "rt_var_cov": ["auditory_z", "visual_z"],  
}
```

```
theta_rho = pm.Beta(  
    "theta_rho",  
    alpha=alpha_theta_rho_correlation,  
    beta=beta_theta_rho_correlation,  
    dims="group",  
)  
rho = pm.Deterministic("rho", 2 * theta_rho - 1, dims="group")
```

- There is
 - One correlation for two variables
 - But one for each group
- Prior on ρ is scaled Beta
 - Beta gives a bounded prior
 - Beta is bounded $[0,1]$
 - ρ is bounded $[-1,1]$



We can use σ and ρ to build a covariance matrix

- Just checking
- `covariance_rows`
 - Uses PyTensor `stack`
 - To build matrixes
 - Is a list of length 2
 - Of PyTensor objects
 - With `dims = ["group", 2]`
- `cov`
 - Uses PyTensor `stack`
 - To combine two 2D matrixes
 - Into a 3D matrix
 - And assigns coordinates

```
audiovisual_coords = {  
    "subject": np.arange(len(audiovisual)),  
    "group": group_order,  
    "rt_var": ["auditory_z", "visual_z"],  
    "rt_var_cov": ["auditory_z", "visual_z"],  
}
```

```
covariance_rows = [  
    pt.stack(  
        [  
            sigma[:, 0] ** 2,  
            sigma[:, 0] * sigma[:, 1] * rho,  
        ],  
        axis=-1,  
    ),  
    pt.stack(  
        [  
            sigma[:, 0] * sigma[:, 1] * rho,  
            sigma[:, 1] ** 2,  
        ],  
        axis=-1,  
    ),  
]  
cov = pm.Deterministic(  
    "cov",  
    pt.stack(covariance_rows, axis=-2),  
    dims=("group", "rt_var", "rt_var_cov"),  
)
```

With μ and cov we can build the likelihood

- Multivariate normal is PyMC distribution
 - With parameters
 - μ : $k \times 1$
 - In our case $k = 1 = \text{rt_var}$
 - cov: $k \times k$
 - Indexing is on the first dimensions by default
 - μ is [group,rt_var]
 - group_index is [subject]
 - So $\mu[\text{group_index}]$ is [subject,rt_var]
 - cov is [group,rt_var,rt_var_cov]
 - group_index is [subject]
 - So $\text{cov}[\text{group_index}]$ is [subject,rt_var,rt_var_cov]

```
audiovisual_coords = {  
    "subject": np.arange(len(audiovisual)),  
    "group": group_order,  
    "rt_var": ["auditory_z", "visual_z"],  
    "rt_var_cov": ["auditory_z", "visual_z"],  
}
```

```
pm.MvNormal(  
    "response_time",  
    mu=mu[group_index],  
    cov=cov[group_index],  
    observed=audiovisual,  
    dims=("subject", "rt_var"),  
)
```

Calculate the difference in correlation in the model

- `delta_rho` : The difference between the correlation in each group

```
rho = pm.Deterministic("rho", 2 * theta_rho - 1, dims="group")
delta_rho = pm.Deterministic(
    "delta_rho",
    rho[musician_index] - rho[nonmusician_index],
)
```

The full bivariate normal model for correlation

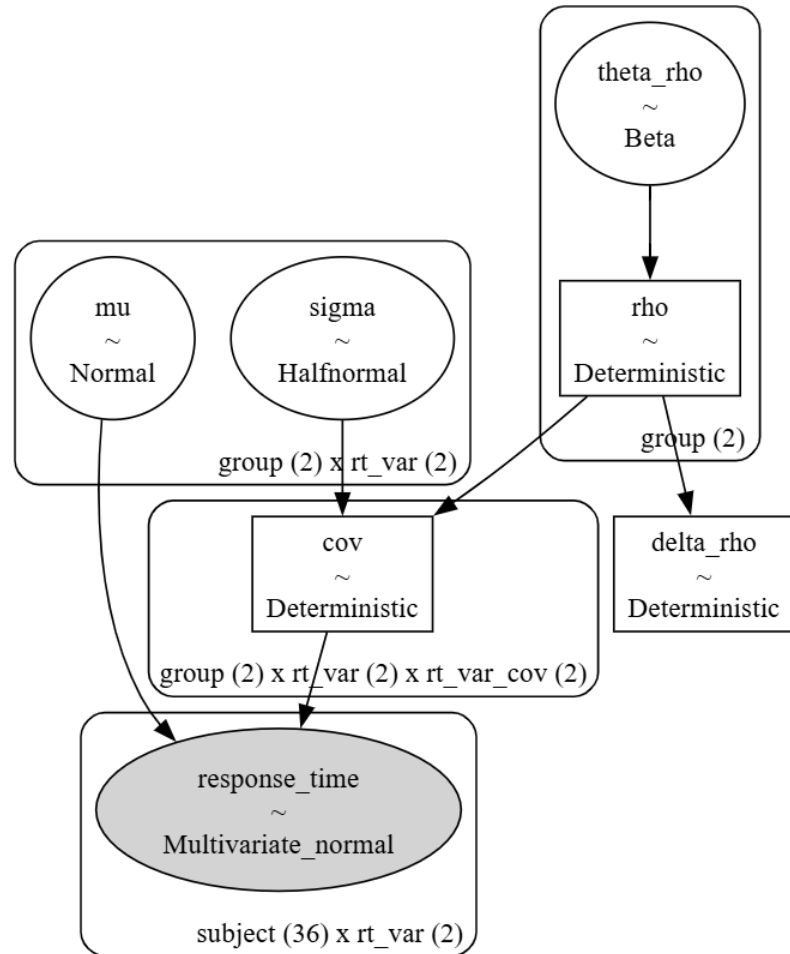
```
with pm.Model(coords=audiovisual_coords) as audiovisual_group_model:
    mu = pm.Normal(
        "mu",
        mu=mu_mu_correlation,
        sigma=sigma_mu_correlation,
        dims=("group", "rt_var"),
    )
    sigma = pm.HalfNormal(
        "sigma",
        sigma=sigma_sigma_correlation,
        dims=("group", "rt_var"),
    )

    theta_rho = pm.Beta(
        "theta_rho",
        alpha=alpha_theta_rho_correlation,
        beta=beta_theta_rho_correlation,
        dims="group",
    )
    rho = pm.Deterministic("rho", 2 * theta_rho - 1, dims="group")
    delta_rho = pm.Deterministic(
        "delta_rho",
        rho[musician_index] - rho[nonmusician_index],
    )
```

```
covariance_rows = [
    pt.stack(
        [
            sigma[:, 0] ** 2,
            sigma[:, 0] * sigma[:, 1] * rho,
        ],
        axis=-1,
    ),
    pt.stack(
        [
            sigma[:, 0] * sigma[:, 1] * rho,
            sigma[:, 1] ** 2,
        ],
        axis=-1,
    ),
]
cov = pm.Deterministic(
    "cov",
    pt.stack(covariance_rows, axis=-2),
    dims=("group", "rt_var", "rt_var_cov"),
)

pm.MvNormal(
    "response_time",
    mu=mu[group_index],
    cov=cov[group_index],
    observed=audiovisual,
    dims=("subject", "rt_var"),
)
```

The graphical version of the model is more compact



$$\mu_{g,v} \sim N(\mu_\mu, \sigma_\mu)$$

$$\sigma_{g,v} \sim \text{HalfNormal}(\sigma_\sigma)$$

$$\rho_g \sim \text{ScaledBeta}(\alpha_\rho, \beta_\rho, \min = -1, \max = 1)$$

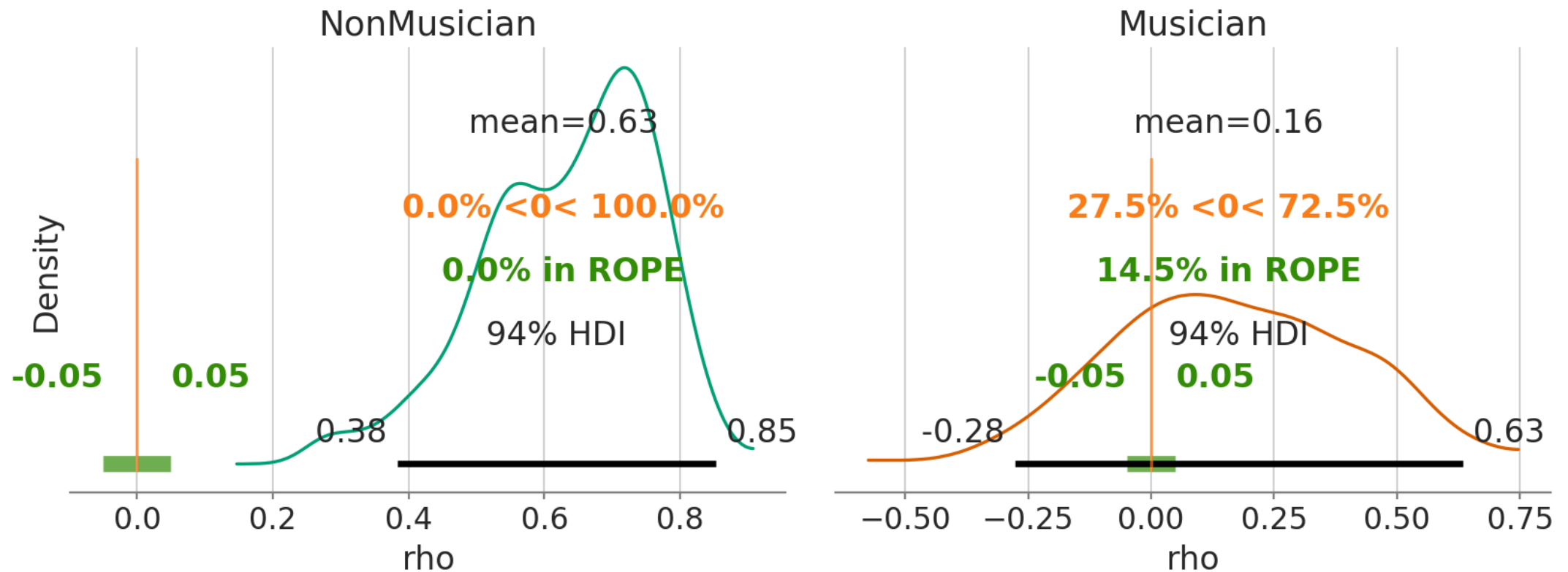
$$\text{cov}_g = \begin{pmatrix} \sigma_{g,1}^2 & \sigma_{g,1}\sigma_{g,2}\rho_g \\ \sigma_{g,1}\sigma_{g,2}\rho_g & \sigma_{g,2}^2 \end{pmatrix}$$

$$RT_i \sim N(\mu_{g(i)}, \text{cov}_{g(i)})$$

$$\Delta\rho = \rho_2 - \rho_1$$

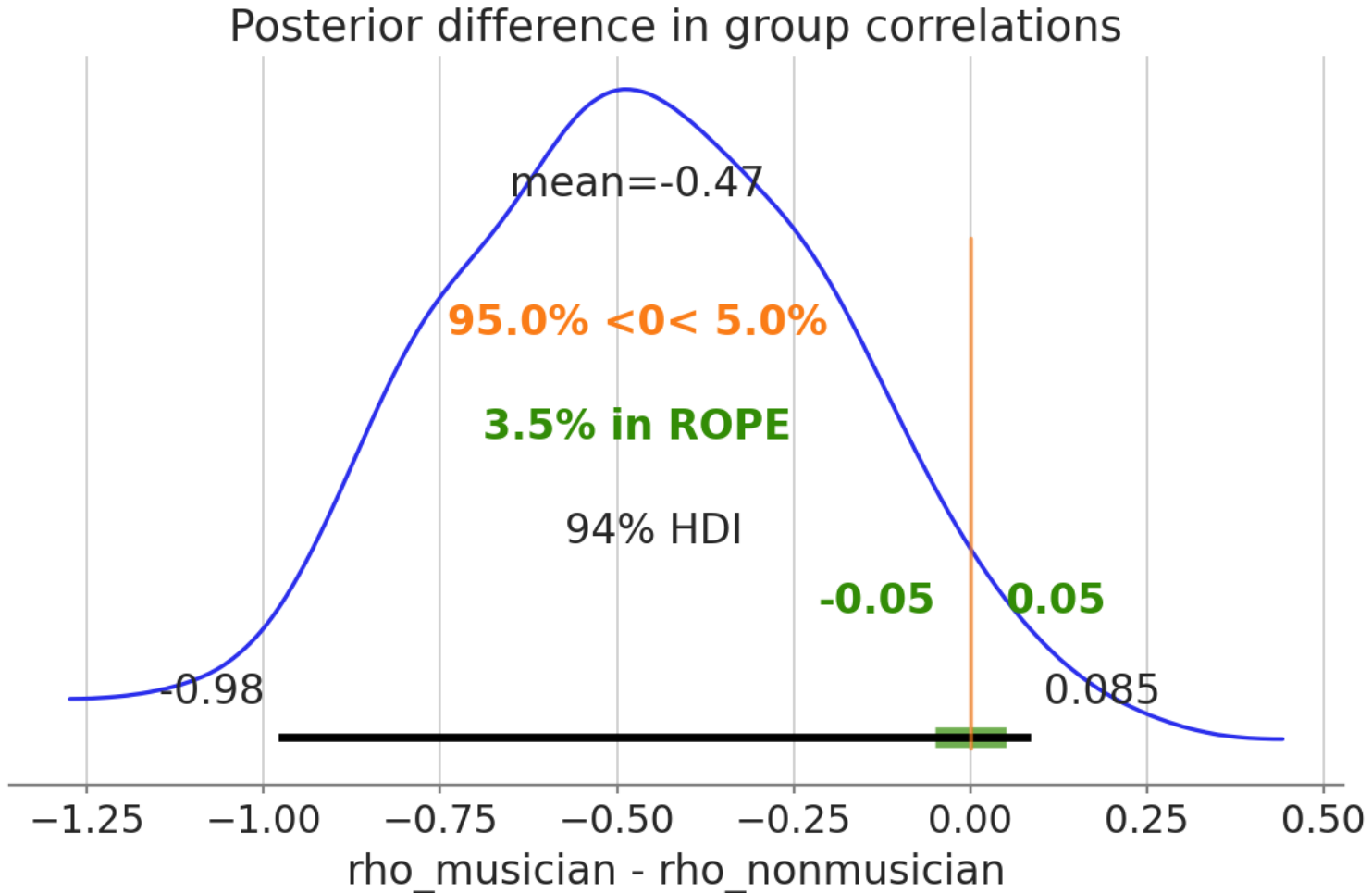
Group-specific ρ posteriors

Posterior group correlations



Using a ROPE of $[-0.05, 0.05]$

$\Delta\rho$: the posterior for the group difference



$$\Delta\rho = \rho_{\text{Musician}} - \rho_{\text{NonMusician}}$$

Posterior mean ≈ -0.48

About 95% of posterior probability below zero

- But HDI overlaps ROPE

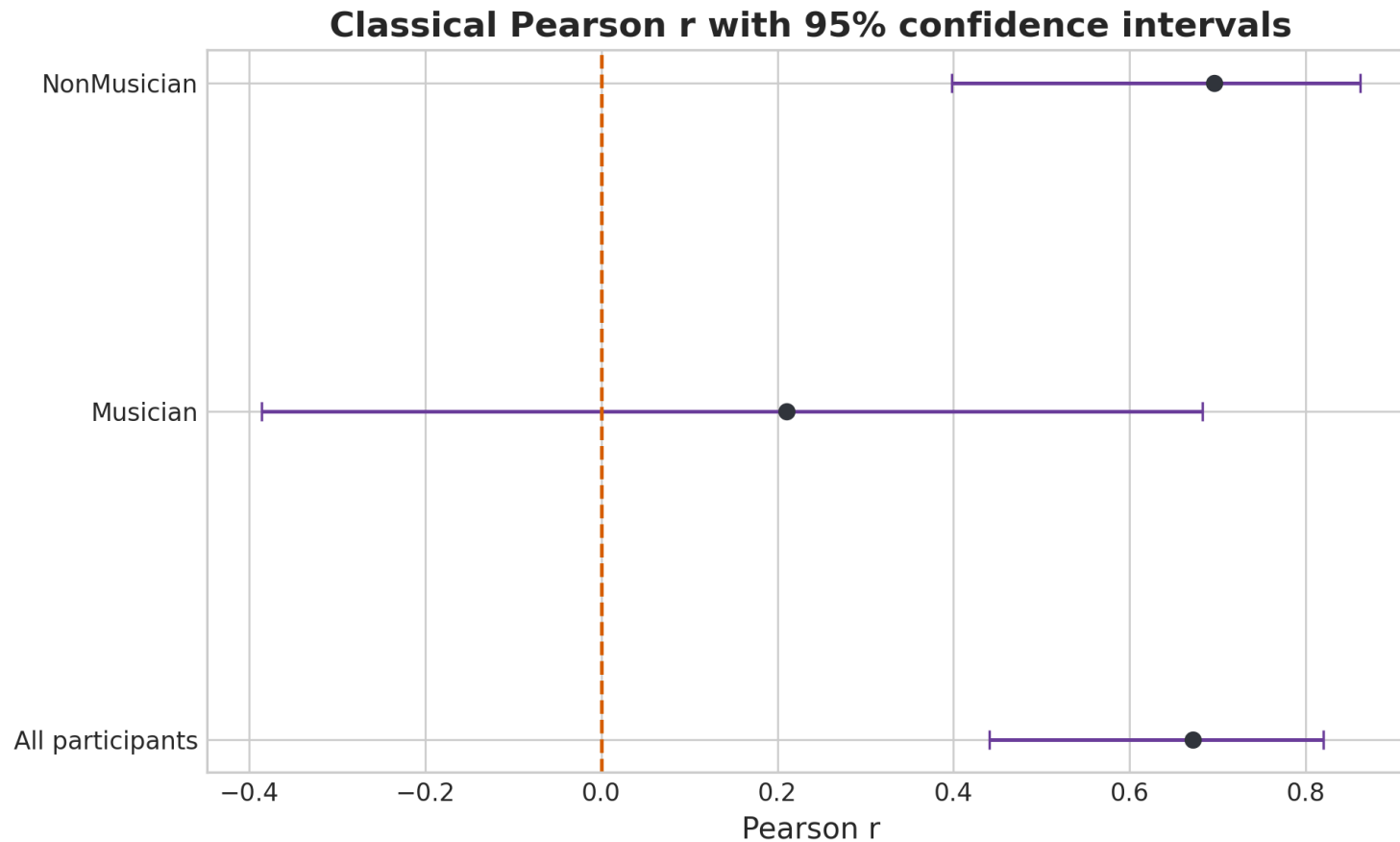
No standard accepted effect size thresholds for $\Delta\rho$

11.3 Frequentist translation

Confidence intervals and Fisher-z tests for correlation differences

Pearson r and Fisher-z confidence intervals

$$r = \hat{\rho}_{MLE} \quad CI: \tanh\left(\operatorname{arctanh}(r) \pm \frac{1.96}{\sqrt{n-3}}\right)$$



Pearson r:

- The standard frequentist estimate of ρ

Fisher-z:

- transform to $\operatorname{arctanh}(r)$,
- Compute Normal CI
- Back-transform

Comparing two independent correlations

$$Z = \frac{\operatorname{arctanh}(r_1) - \operatorname{arctanh}(r_2)}{\sqrt{\frac{1}{n_1 - 3} + \frac{1}{n_2 - 3}}}$$

- Frequentist question
 - Are ρ_1 and ρ_2 significantly different?
- Standard test
 - Fisher-z difference test

	sample	n	pearson_r	ci_2.5%	ci_97.5%	p_value_H0_rho_0
0	All participants	36	0.672	0.441	0.819	0.00
1	Musician	13	0.211	-0.385	0.682	0.49
2	NonMusician	23	0.696	0.398	0.861	0.00

Go to wooclap.com and use the code **NEMGOJY**



Which of the following does not use the t distribution for frequentist statistical testing



- ① Significance of the difference of 2 means
- ② Significance of a correlation
- ③ Significance of the difference between correlation coefficients
- ④ Significance of a regression model

Click on the projected screen to start the question

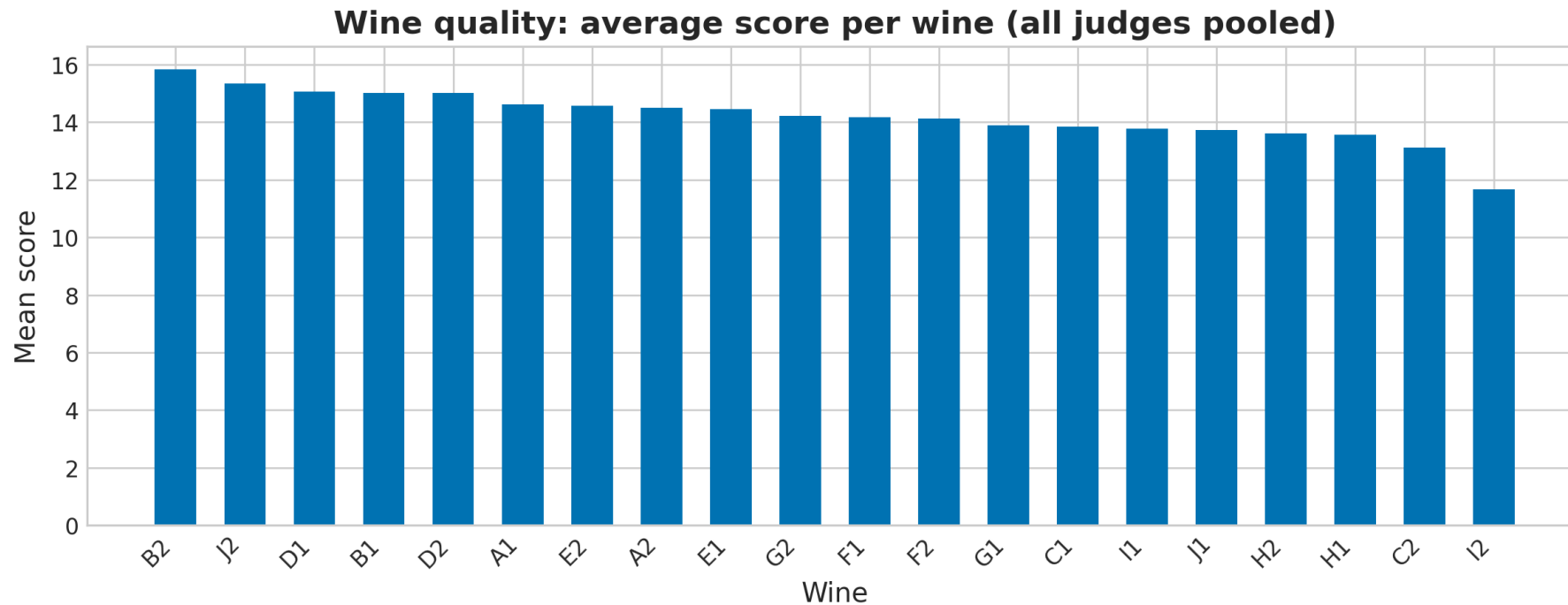


11.4 Repeated measurements: wine ratings

From paired scores to hierarchical structure in judge data

The Wines2012 data: wines and judges

9 judges, 20 wines, 180 ratings: every judge scores every wine



Trying to figure out which wines are better than which other wines

A first model: scores vary by wine only

$$S_{ij} = N(\alpha + Q_i, \sigma) \quad \sum_i Q_i = 0$$

i : for wine

j : for judge

- Simplest model: each score depends only on wine identity
 - All ratings for a given wine treated as exchangeable
 - Zero-sum constraint: wine effects are deviations from the grand mean

The model uses the **ZeroSumNormal** distribution

$$S_{ij} = N(\alpha + Q_i, \sigma) \quad \sum_i Q_i = 0$$

i : for wine

j : for judge

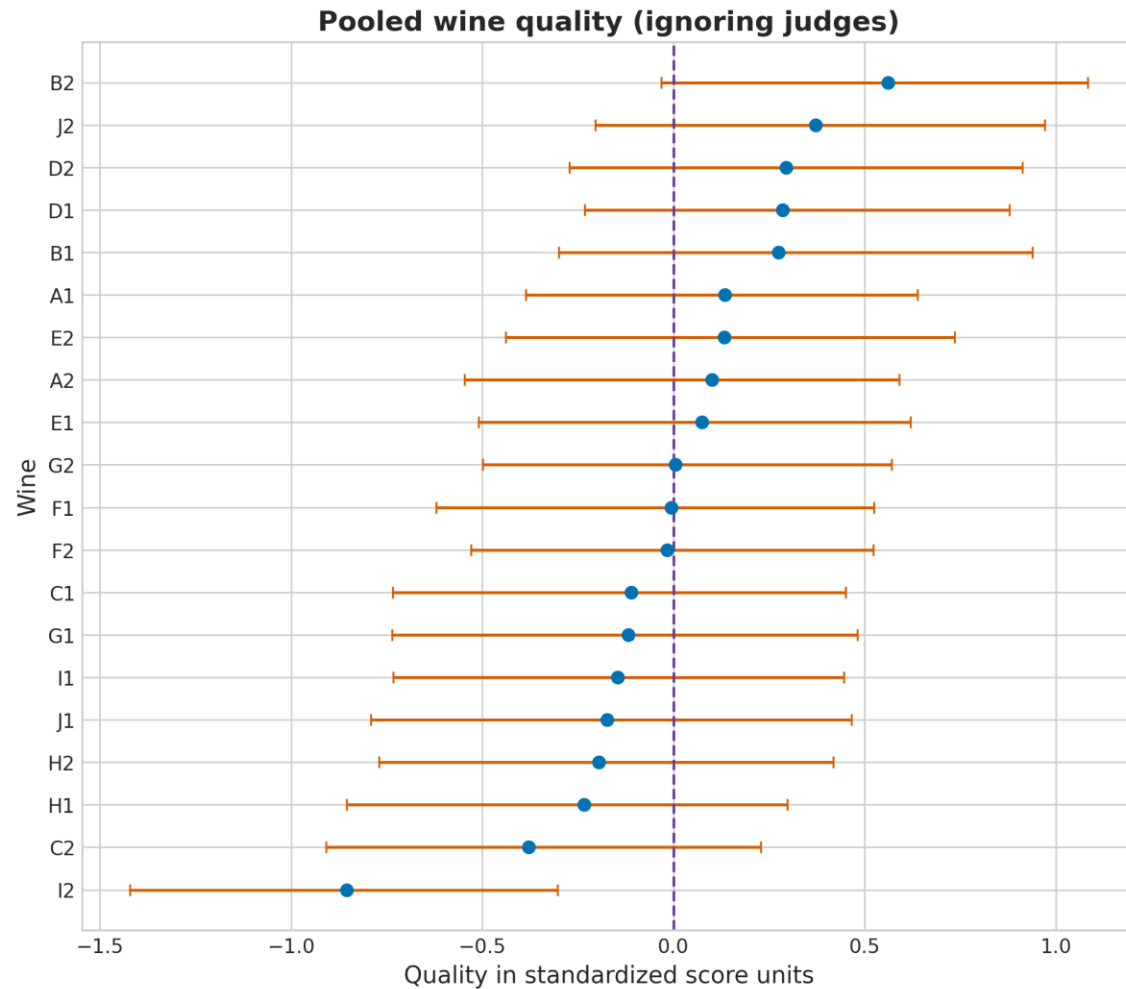
```
with pm.Model(coords=wine_coords) as wine_model:
    alpha_pooled = pm.Normal("alpha", mu=mu_alpha, sigma=sigma_alpha)
    quality = pm.ZeroSumNormal("quality", sigma=sigma_mu_quality, dims="wine")
    sigma = pm.HalfNormal("sigma", sigma=sigma_sigma)
    mu = pm.Deterministic(
        "mu",
        alpha + quality[wine_idx],
        dims="obs",
    )
    pm.Normal("score", mu=mu, sigma=sigma, observed=score_z, dims="obs")
```

- **ZeroSumNormal**

- N variables
 - Normally distributed
 - Sum to 0

- Useful for categorical regression models

Pooled model quality estimates

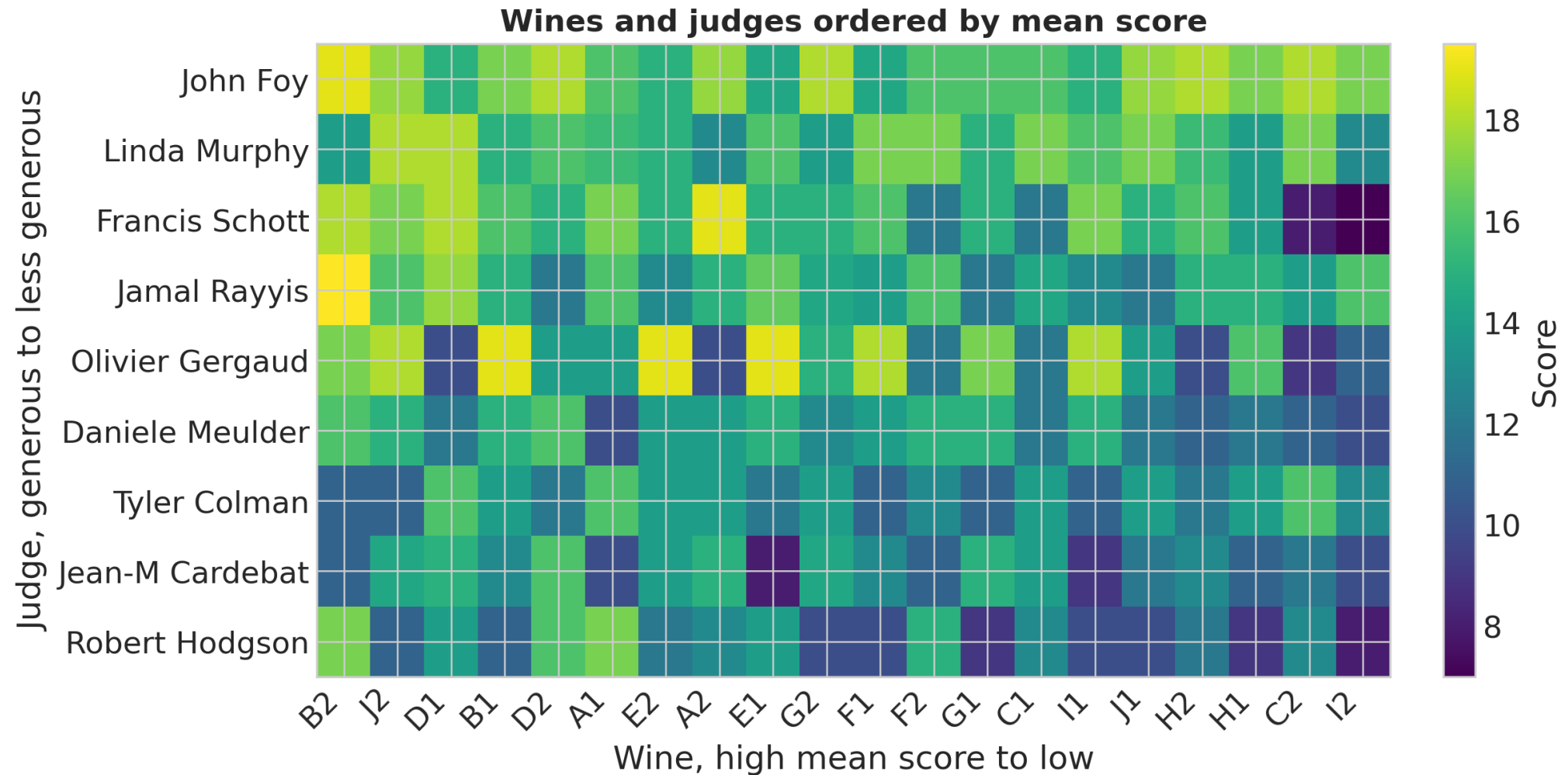


Rankings visible; some wines clearly above or below average

11.5 Wine quality and judge generosity

Separating latent wine quality from judge-specific generosity

The heatmap reveals judge generosity



Rows differ systematically even after wines are ordered.

New model: wine quality + judge generosity

$$S_{ij} = \alpha + Q_i + G_j + \varepsilon_{ij}$$
$$\sum_i Q_i = 0 \qquad \sum_j G_j = 0$$

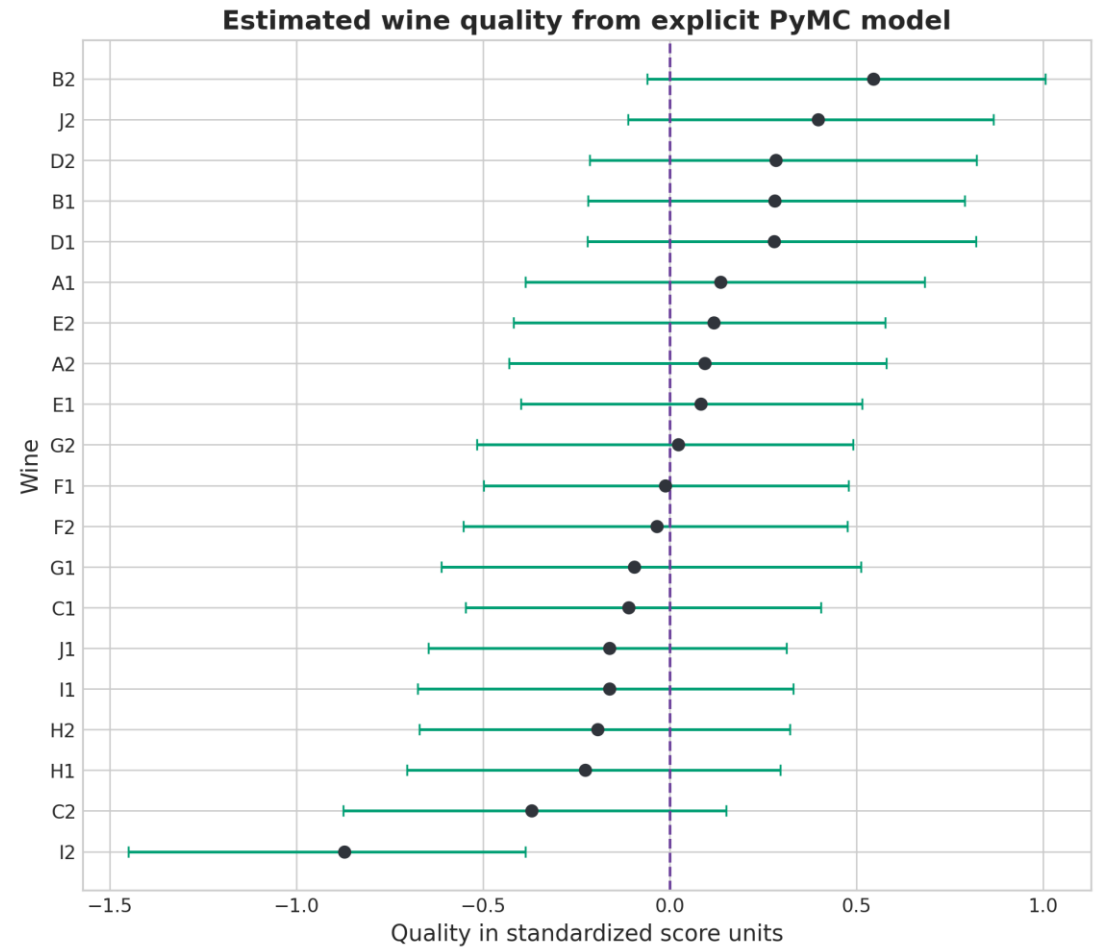
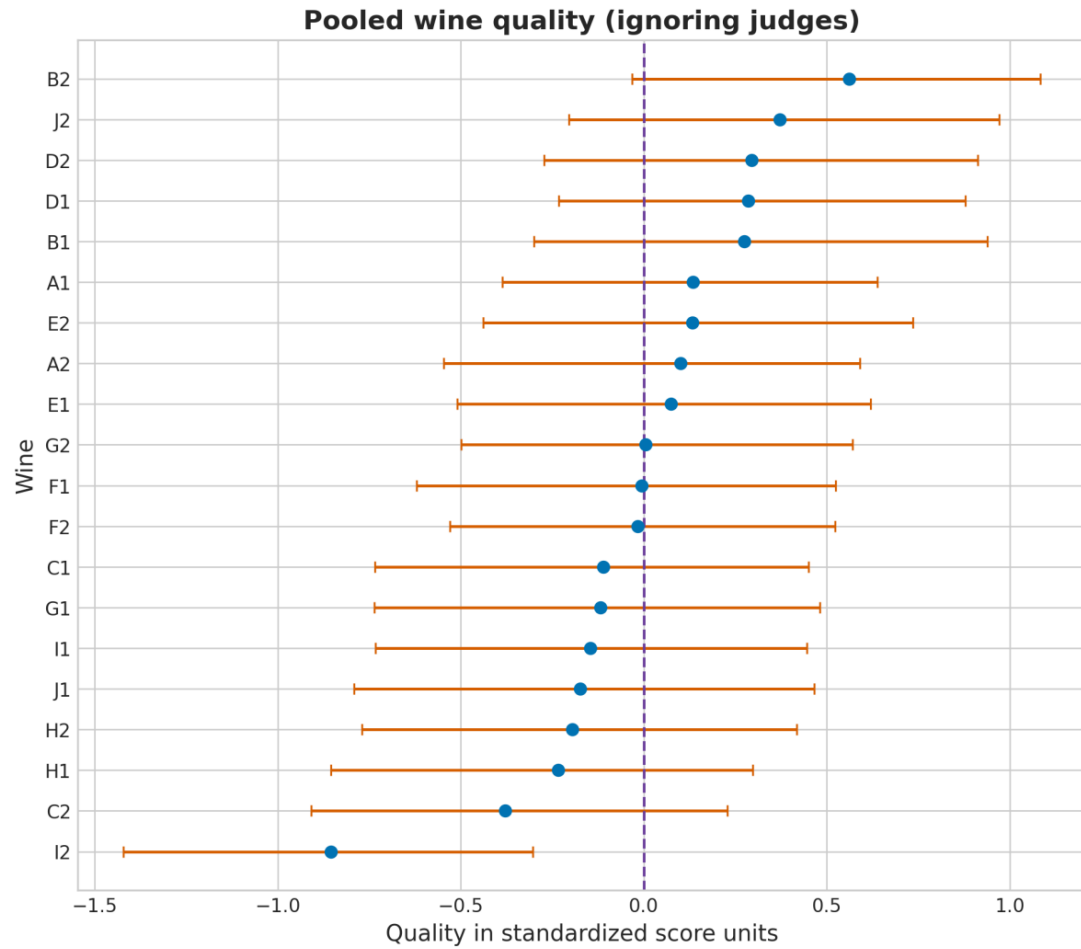
- Hierarchical in judges
 - Information sharing
- Non-hierarchical in wines
 - Differences in wines is what we're interested in
 - Don't want shrinkage

```
with pm.Model(coords=wine_coords) as wine_repeated_model:
    alpha = pm.Normal("alpha", mu=mu_alpha, sigma=sigma_alpha)
    quality = pm.ZeroSumNormal("quality", sigma=sigma_mu_quality, dims="wine")

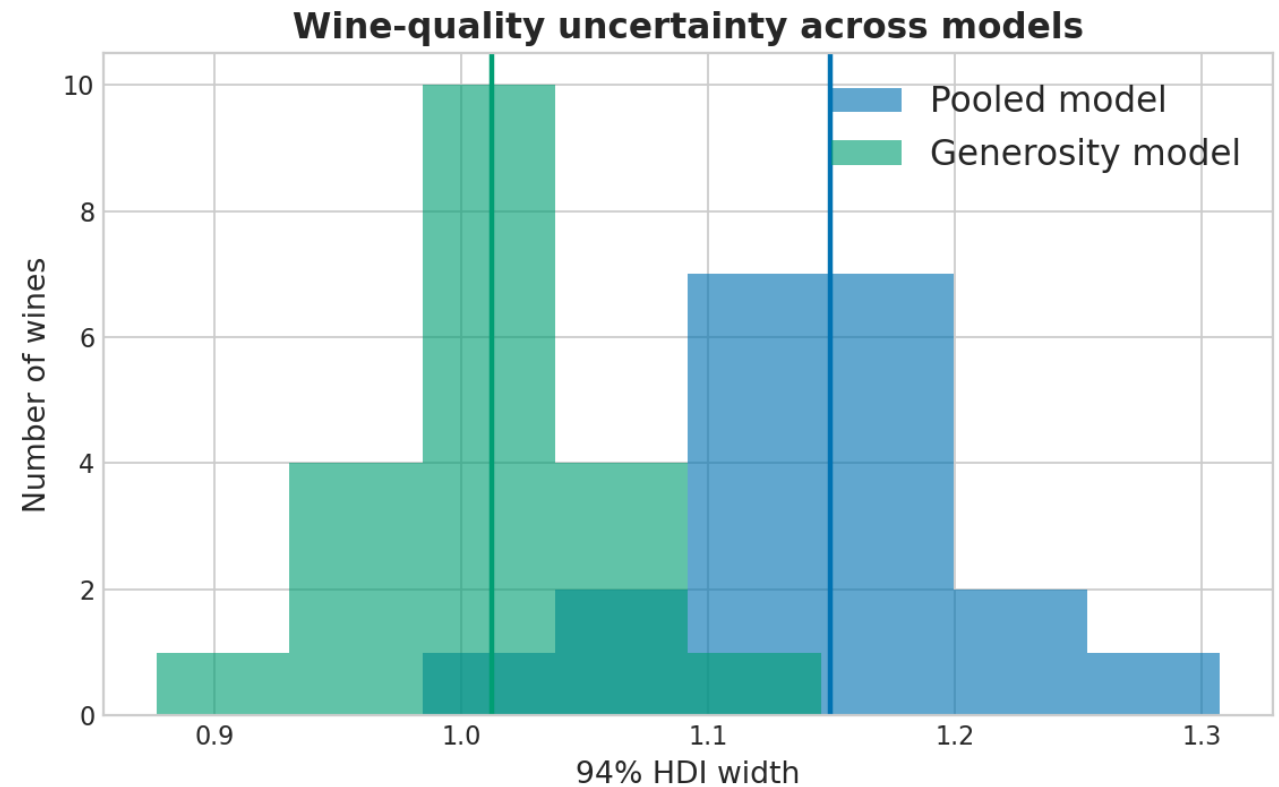
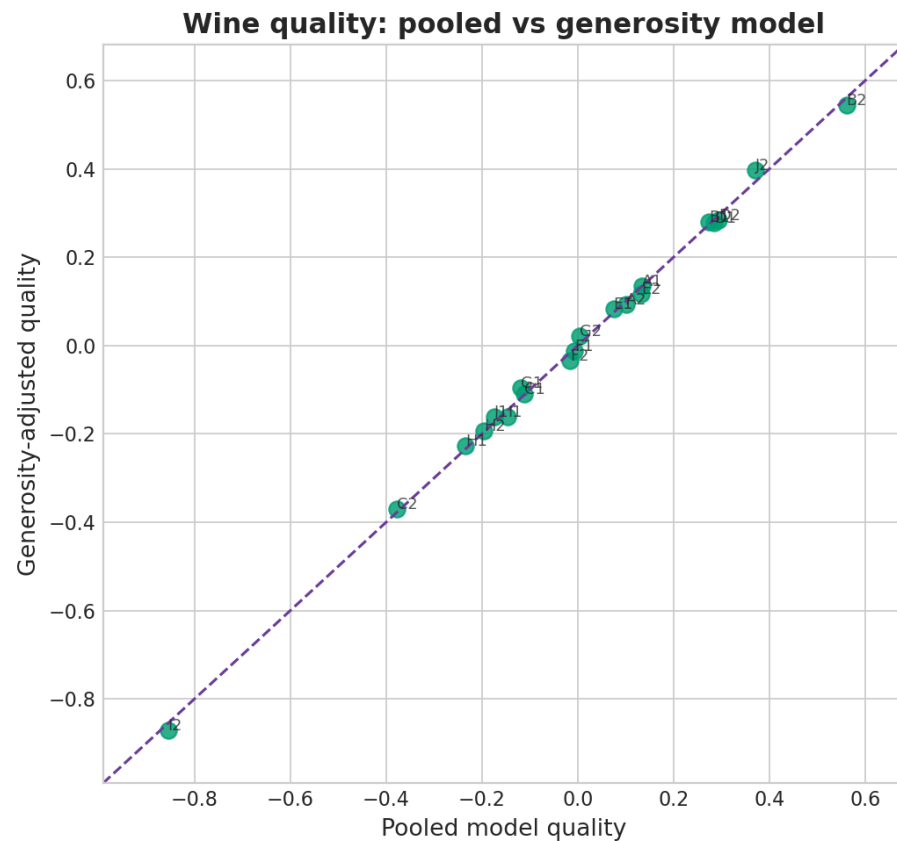
    sigma_judge = pm.HalfNormal("sigma_judge", sigma=sigma_sigma_judge)
    generosity = pm.ZeroSumNormal("generosity", sigma=sigma_judge, dims="judge")

    sigma = pm.HalfNormal("sigma", sigma=sigma_sigma_score)
    mu = pm.Deterministic(
        "mu", alpha + quality[wine_idx] + generosity[judge_idx], dims="obs",
    )
    pm.Normal("score", mu=mu, sigma=sigma, observed=score_z, dims="obs")
```

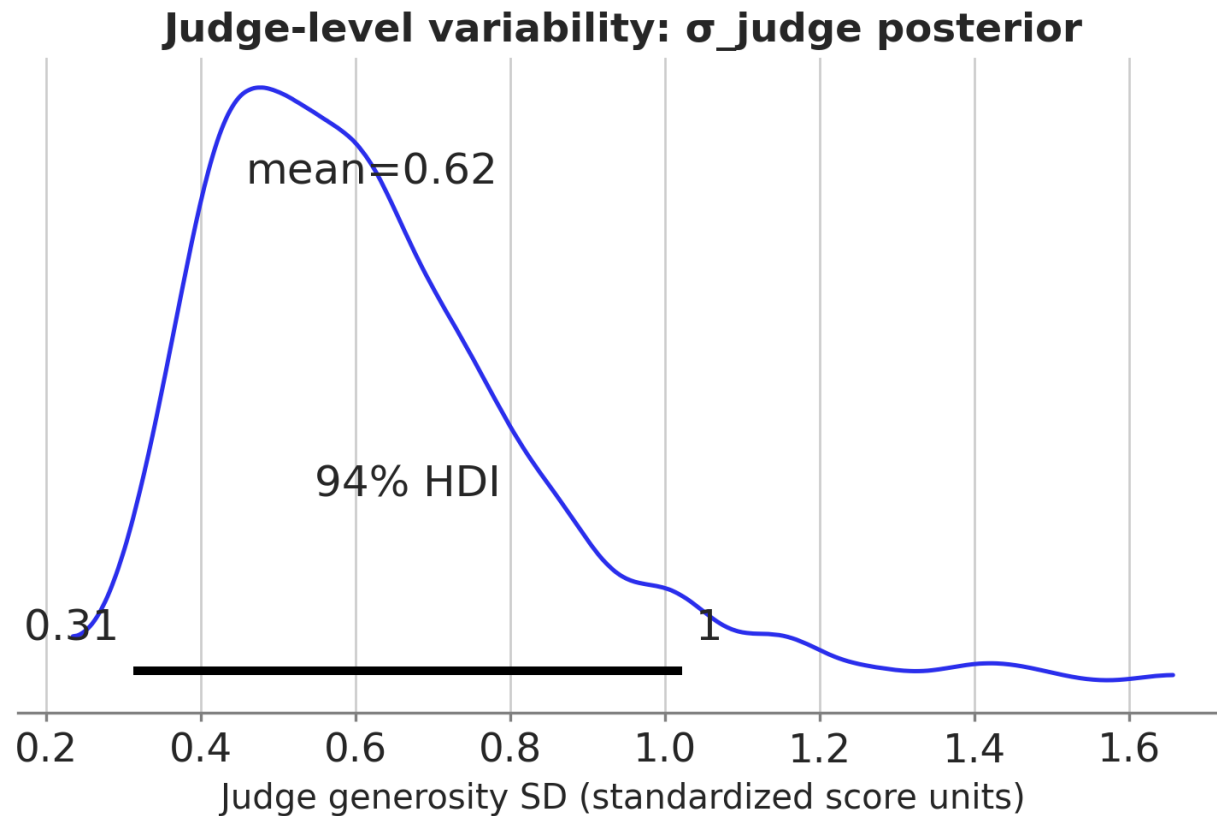
Wine quality posterior seems similar



Estimates now have less uncertainty



Differences between judges could matter



$$G_j \sim \text{Normal}(0, \sigma_{\text{judge}})$$
$$\sigma_{\text{judge}} \sim \text{HalfNormal}(\dots)$$

σ_{judge} : population standard deviation of judge generosity

If posterior mass is away from zero $\rightarrow$ judges differ substantially

This matters if judges judge different wines

11.6 Fixed and random effects

Fixed effects are ones where levels matter, random effects only the spread matters

Fixed and random effects

- A frequentist concept
 - Borrowed into Bayesian analysis
- Frequentist framing:
 - Fixed effects
 - An effect we care about
 - A level in a factor that will not change in a new experiment
 - A factor for which we may do hypothesis tests
 - Random effect
 - A source of noise
 - A factor with replaceable levels
 - A factor for which we will not do hypothesis tests
- Bayesian translation:
 - Fixed effects
 - Regression factors with fixed priors
 - Estimate the levels
 - Random effects
 - Regression factors with hierarchical variance
 - Estimate the variance

```
quality = pm.ZeroSumNormal("quality", sigma=sigma_mu_quality, dims="wine")

sigma_judge = pm.HalfNormal("sigma_judge", sigma=sigma_sigma_judge)
generosity = pm.ZeroSumNormal("generosity", sigma=sigma_judge, dims="judge")
```

Wilkinson notation for random effects

```
score_z ~ 0 + wine + (1 | judge)
```

- `0 + wine` : one wine-specific term per wine identity (fixed-effect-like)
- `(1 | judge)` : varying intercept per judge; Bambi estimates judge-level spread from data
- Bambi recovers the same wine quality estimates as the explicit PyMC model
- Same notation used in frequentist `lme4` / `nlme`: shared grammar across frameworks

Frequentist fitting of complex models

- Statsmodel uses formulas in its `mixedlm` function
- Various fitting formulas available
 - Safest to use `reml=True`
- Analyzing these models is beyond the scope of the course

```
wine_mixedlm_formula = "score_z ~ 0 + wine"
wine_mixedlm_fit = smf.mixedlm(
    wine_mixedlm_formula,
    data=wines,
    groups="judge",
    re_formula="1",
).fit(reml=True)
```

	wald_chi2	p_value	df
wine	26.9989	0.1353	20

	quantity	value	interpretation
0	fixed_wine_SD	0.3306	SD of fitted wine means, in score_z units

< Select another question

Edit question

Go to wooclap.com and use the code **NEMGOJY**



Which of the following seems like it would not make sense to treat it as a random effect?



- ① The differences between participants in a psychology experiment
- ② The differences between different schools in a study
- ③ The differences between classrooms in an educational study
- ④ The differing sensitivity of different MRI machines in different hospitals

Click on the projected screen to start the question

wooclap



100%



11.7 Frequentist multi-factor models

Partial and full F for hypothesis testing

Frequentist multi-factor models

- We've seen models like these before
- Fit with “ordinary least squares”
- Produce an “ANOVA table”

```
wine_ols_formula = "score_z ~ wine + judge"  
wine_ols_fit = smf.ols(wine_ols_formula, data=wines).fit()
```

	model	F	df_model	df_resid	p_value	R2	adj_R2
0	score_z ~ wine + judge	3.522	27.0	152.0	0.0	0.3849	0.2756

	SS	df	MS	F	p_value
wine	19.5585	19.0	1.0294	1.4210	0.1245
judge	49.3296	8.0	6.1662	8.5119	0.0000
Residual	110.1118	152.0	0.7244	NaN	NaN

Calculating F for the model

- The F statistic distributes with the F distribution
 - Under the null hypothesis
- k is the number of variables in the data matrix
 - Each regression coefficient
 - Each dummy variable

$$F = \frac{MS_{reg}}{MS_{res}} = \frac{\frac{SS_{reg}}{df_{reg}}}{\frac{SS_{res}}{df_{res}}} \sim F(df_{reg}, df_{res})$$

$$SS_{reg} = \sum_{i=1}^N (\hat{y}_i - \bar{y})^2$$

$$SS_{res} = \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$$df_{reg} = k$$

$$df_{res} = N - (k + 1)$$

20 wines + 9 judges = 19 + 8 dummy variables
 $k = 17$

Calculating F for a factor

- The F statistic distributes with the F distribution
 - Under the null hypothesis that the factor has no effect
- l is the number of dummy variables in the factor
- $\hat{y}_{factor}$ is the predicted value for y without the factor

$$F = \frac{MS_{factor}}{MS_{res}} = \frac{\frac{SS_{factor}}{df_{factor}}}{\frac{SS_{res}}{df_{res}}} \sim F(df_{factor}, df_{res})$$

$$SS_{factor} = \sum_{i=1}^N \left(\hat{y}_{factor,i} - \hat{y}_i \right)^2$$

$$SS_{res} = \sum_{i=1}^N \left(y_i - \hat{y}_i \right)^2$$

$$df_{factor} = l$$

$$df_{res} = N - (k + 1)$$

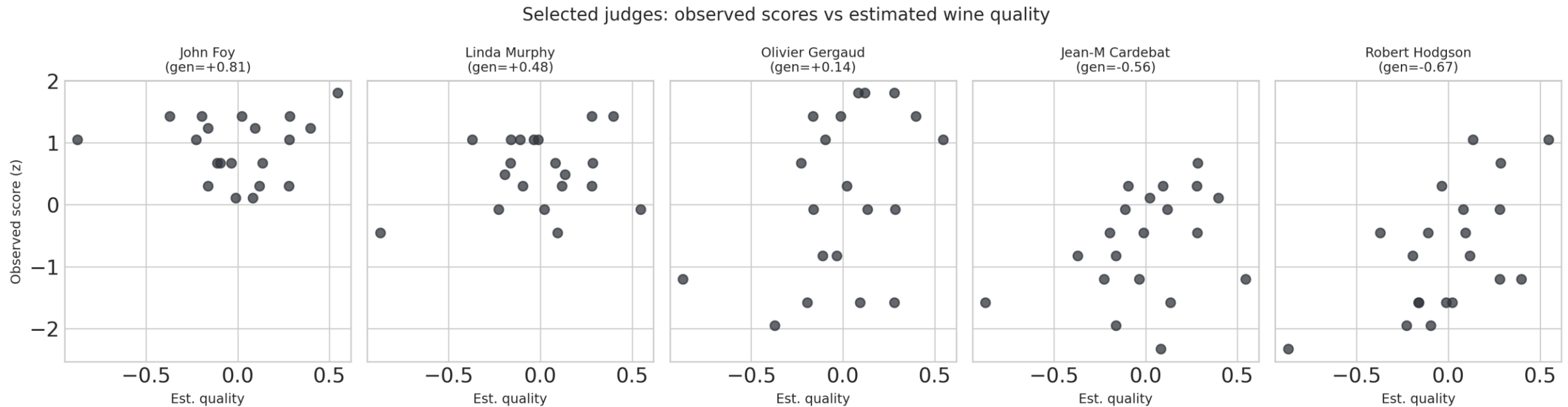
20 wines : 19 dummy variables
9 judges: 8 dummy variables

11.6 Discrimination: motivation and model

Modeling judge sensitivity to latent quality, not just intercepts

Some judges separate wine quality more clearly than others

Each panel is one judge; a stronger relationship between estimated quality and score means higher discrimination.



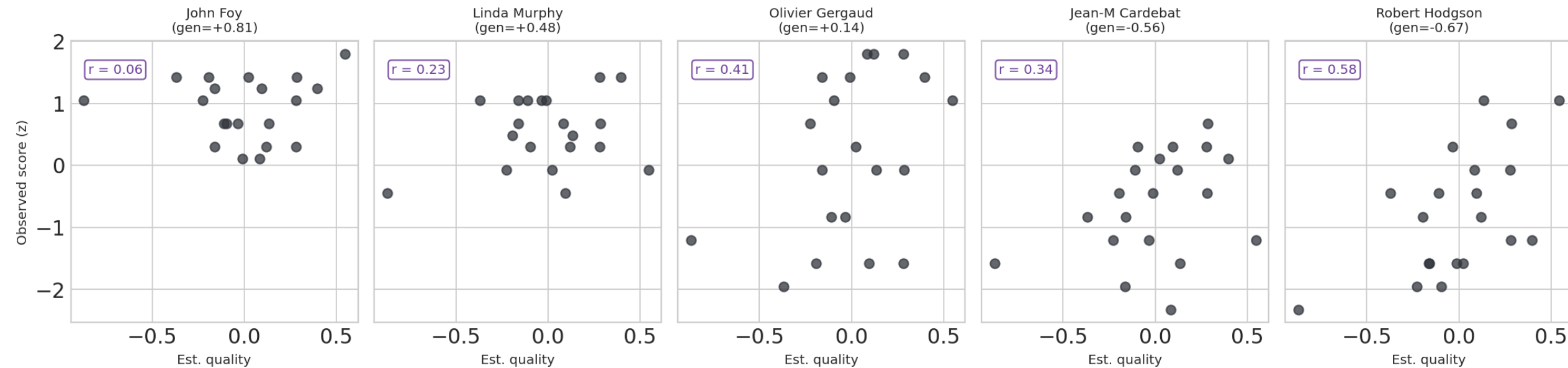
Generosity is a level shift. Discrimination is the strength of the quality-score relationship.

This motivates letting judges vary in sensitivity, not just in average score.

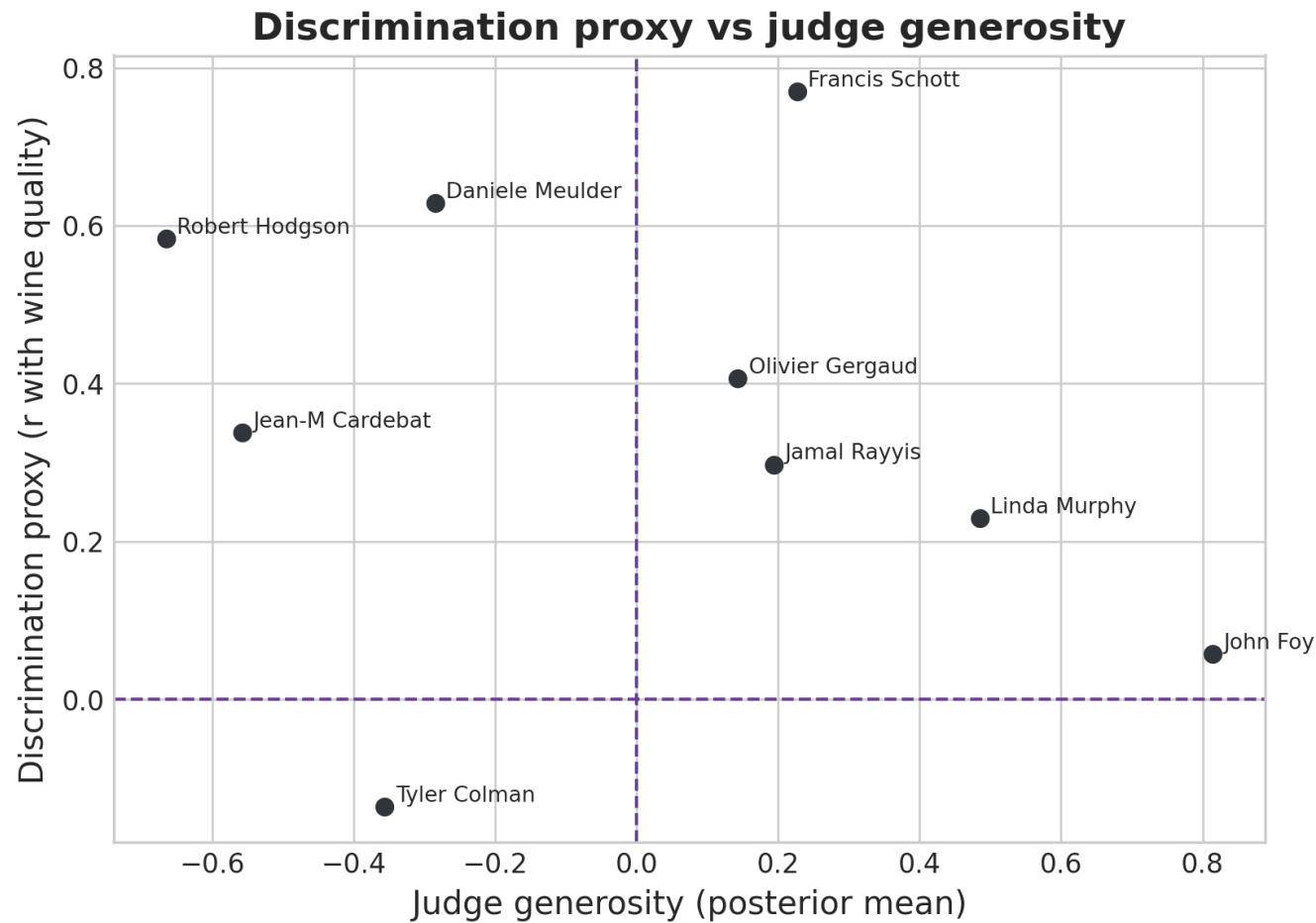
Discrimination can be estimated by correlation

How well does this judge's score correlate with estimated wine quality

Selected judges: observed scores vs estimated wine quality



Discrimination proxy vs generosity



Discrimination vs judge generosity:

- There may be a relationship

The discrimination model lets sensitivity vary by judge

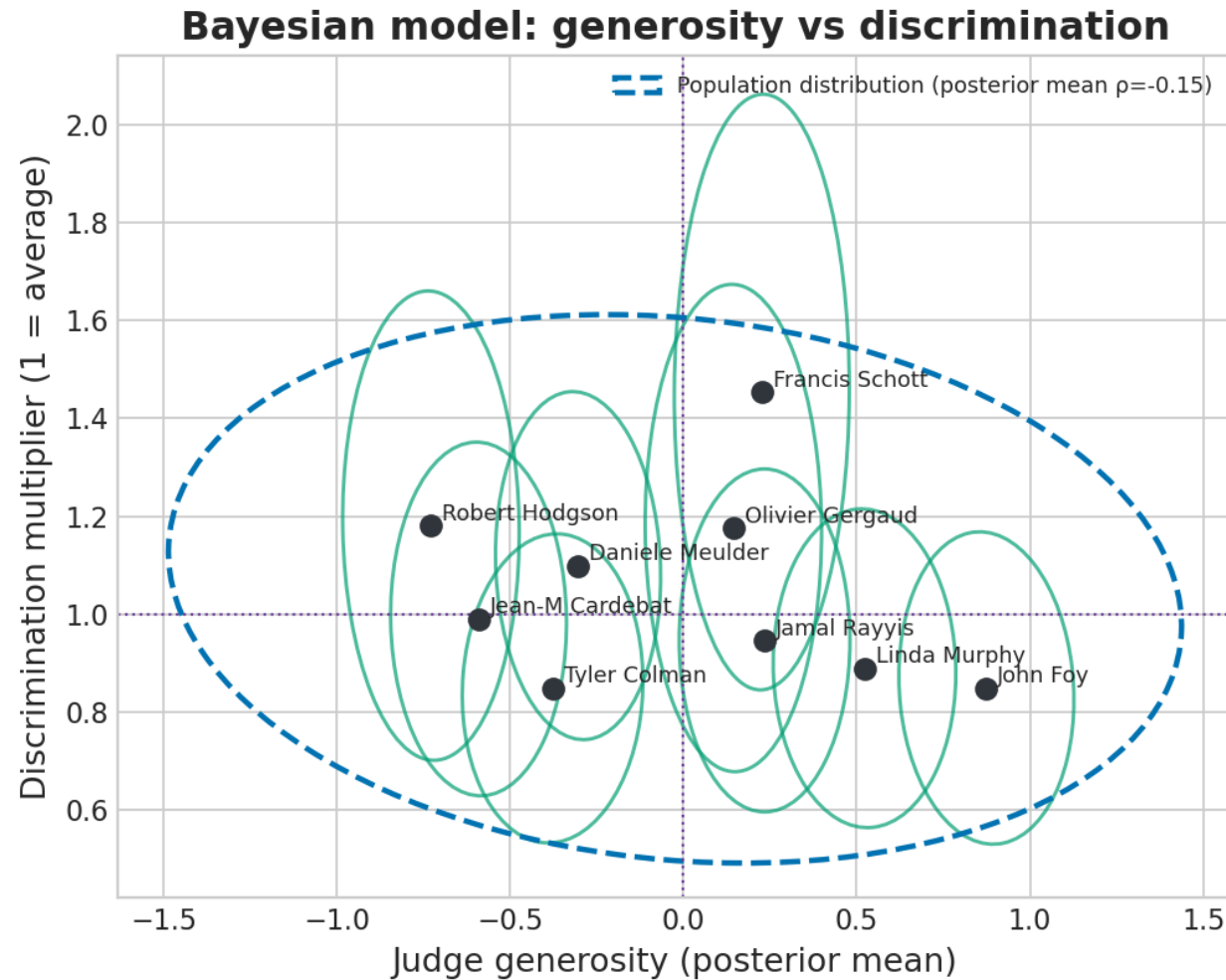
$$(G_j, L_j) \sim \text{MvNormal}(0, \Sigma_{\text{judge}})$$
$$\mu_{ij} = G_j + D_j Q_i, \quad D_j = \exp(L_j) > 0$$

- Q_i : latent wine quality, inferred inside the model
- G_j : judge generosity; shifts all scores up or down
- $D_j = \exp(L_j)$: positive sensitivity to latent quality Q_i
- (G_j, L_j) : modeled jointly, so their correlation can be estimated

```
quality = pm.Normal(  
    "quality",  
    mu=mu_mu_quality,  
    sigma=sigma_mu_quality,  
    dims="wine",  
)  
judge_effects = pm.MvNormal(  
    "judge_effects",  
    mu=judge_effect_mu,  
    cov=judge_effect_cov,  
    dims=("judge", "judge_effect"),  
)  
generosity = judge_effects[:, 0]  
log_discrimination = judge_effects[:, 1]
```

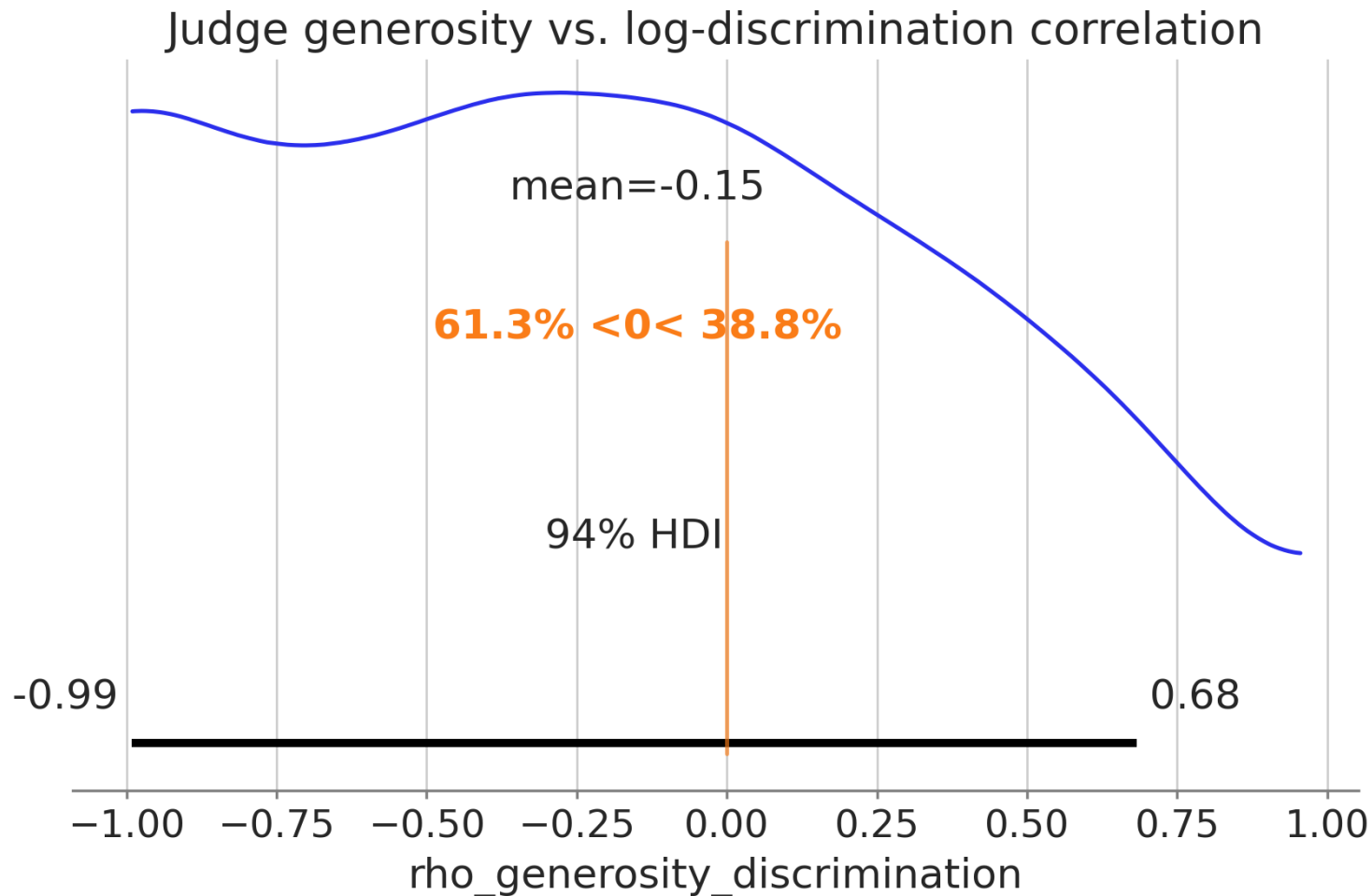
```
mu = pm.Deterministic(  
    "mu",  
    generosity[judge_idx] + discrimination[judge_idx] * quality[wine_idx],  
    dims="obs",  
)  
pm.Normal("score", mu=mu, sigma=sigma, observed=score_z, dims="obs")
```

Generosity vs discrimination: exploratory to Bayesian



Bayesian posterior: per-judge generosity vs discrimination with 1.5σ uncertainty ellipses.
Dashed ellipse = fitted population distribution, not an individual judge posterior.

Exploratory suggestion vs model uncertainty



Posterior for $\rho(G, L)$: mean ≈ -0.15 , 94% HDI $\approx (-0.99, 0.68)$ — extremely wide

$\Pr(\rho < 0) \approx 0.61$: very weak evidence for a negative relationship

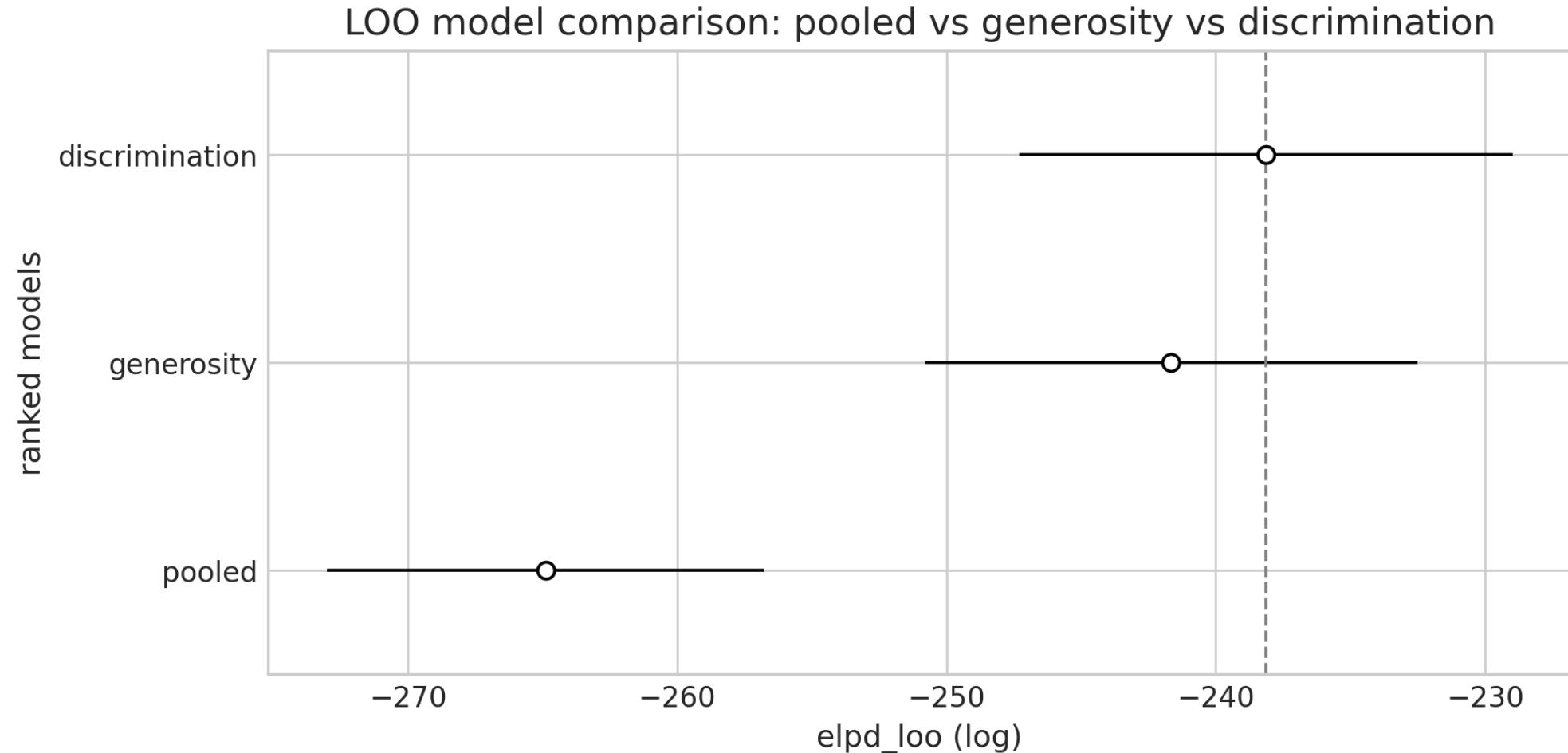
With only 9 judges, the model cannot resolve this latent correlation

11.7 Three-model comparison

- Model 1 (Pooled): scores vary by wine only
- Model 2 (Generosity): wine + judge generosity (random intercepts)
- Model 3 (Discrimination): wine + judge generosity + discrimination (random slopes on latent quality)

Is the added complexity justified by improved predictive performance?

LOO results: which model predicts best?



Generosity model clearly outperforms pooled — judges differ meaningfully.

Discrimination adds little or uncertain improvement over generosity — LOO intervals overlap.

< Select another question

Edit question

Go to wooclap.com and use the code **NEMGOJY**



What would you conclude from this figure?

- 1 All three models are good at explaining the data
- 2 The discrimination model is the best because it has the lowest elpd_loo and the highest R-squared value
- 3 The generosity model should be preferred because it is the simplest and has the lowest elpd_loo
- 4 The discrimination and generosity models have similar elpd_loo and they make similar predictions

Click on the projected screen to start the question

wooclap



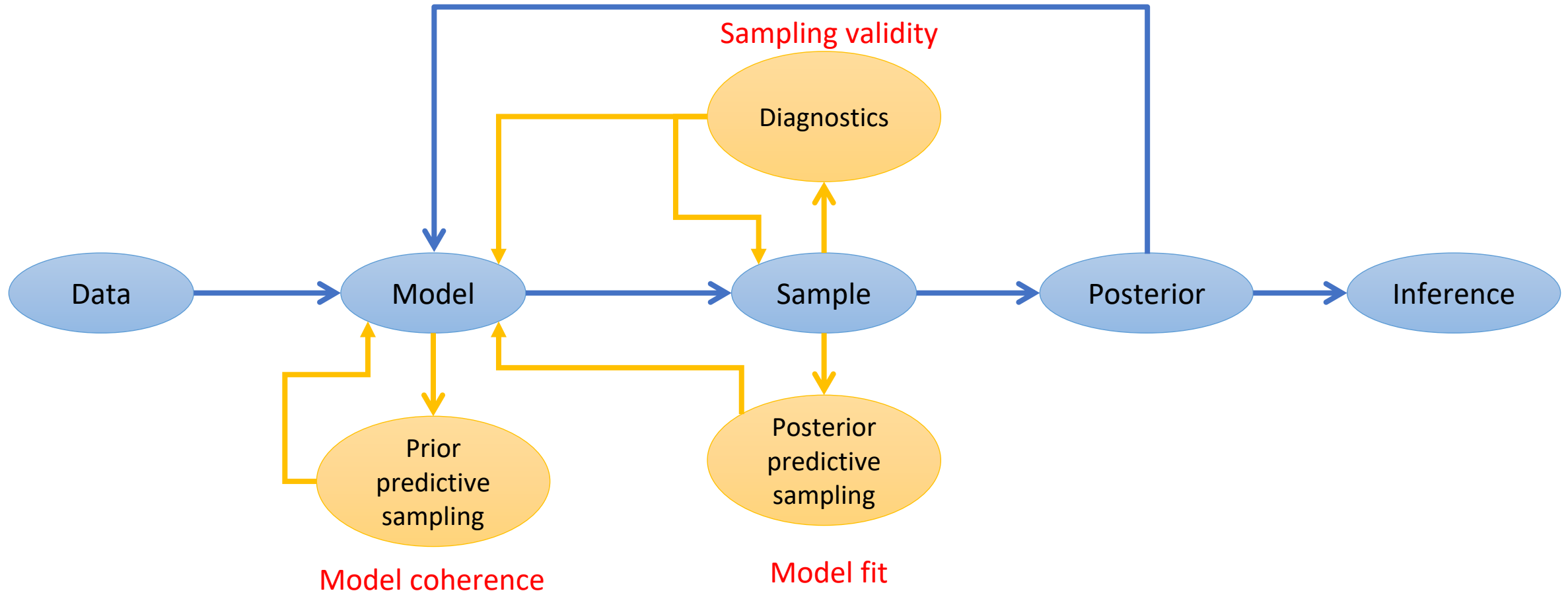
100%



11.8 Next time

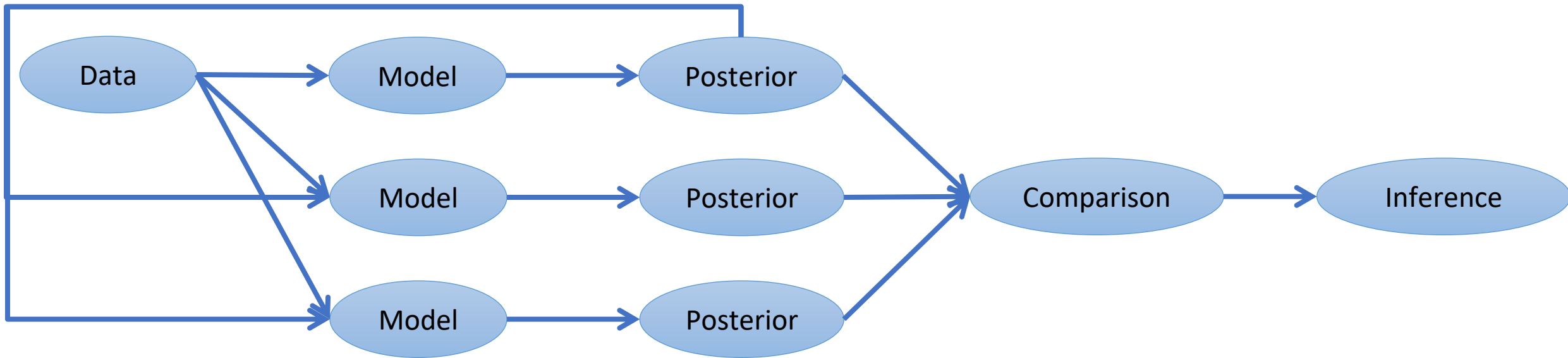
What changes when the data problem starts before the model

The Bayesian workflow has become natural and obvious



The same workflow now covers many research designs

- We have learned:
 - Varieties of models
 - Model development
 - Model comparison
 - Model selection



Before the model, the data can already mislead

So far we have mostly assumed well-collected, representative data.

Target population → sample → measurements → observed data → model

What can go wrong upstream:

- Measurement: are we measuring what we think we are?
- Sampling: does the sample represent the target population?
- Missingness: are missing data informative (MNAR)?
- Bias and confounding: treatment effects versus selection effects

Next time: how analyses can be led astray before or during modeling.